Assignment 13.2

R.Ashmitha Reddy

2303A52111

Batch 40

Task Description -1 (Refactoring – Eliminating Repetitive Logic) •

Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

Sample Code:

length = 8 width = 4 print("Area:", length * width) print("Perimeter:", 2 * (length + width)) length = 12 width = 6 print("Area:", length * width) print("Perimeter:", 2 * (length + width))

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Prompt and Code

```
# transform the following repeated logic code into reusable functions to calculate the area and perimeter of a rectangle.
def calculate_area(length, width):
    return length * width

def calculate_perimeter(length, width):
    return 2 * (length + width)
length = 10
width = 5
print("Area of rectangle:", calculate_area(length, width))
print("Perimeter of rectangle:", calculate_perimeter(length, width))
```

Task Description -2 (Refactoring – Modularizing Inline Calculations)

- Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

- Identify repeated calculation patterns in the code.
- Convert them into reusable functions.
- Ensure readability and proper documentation.

Sample Code: amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000 discount = amount *

0.10 final_amount = amount -

discount print("Final Amount:",

final_amount)

Expected Output:

A modular Python program that uses a reusable function to compute

final amounts for different inputs.

Prompt and Code:

```
# convert the following into reusable functions by extracting repeated inline code
def calculate_final_amount(amount):
    discount = amount * 0.10
    return amount - discount

print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```

Task Description -3 (Refactoring – Improving Loop and

Conditional Performance)

- Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

- Use AI to recommend more efficient data structures or logic simplifications.
- Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Sample Code:

students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"] for

name in check_list:

exists = False for

student in students: if

name == student:

exists = True if

exists:

print(name, "found") else:

print(name, "not found")

Expected Output:

An optimized Python program that performs faster lookups using

efficient logic, along with execution performance comparison.

Prompt and Code:

```
# refactor the following code to use a function that checks if a name exists in a list of students, and then use that function to check for multiple names.
def is_student_present(students_list, name):
    return name in students_list

students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
    if is_student_present(students, name):
        print(name, "found")
    else:
        print(name, "not found")
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with

Constants)

• Task: Utilize AI assistance to improve maintainability by replacing

hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing

results.

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100) print("Simple

Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for

clearer and maintainable calculations..

Prompt and Code

```
# improve maintainability by replacing hardcoded numerical values with named constants.
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 2
TIME = 5
print("Simple Interest:", (PRINCIPAL_1 * RATE * TIME) / 100)
print("Simple Interest:", (PRINCIPAL_2 * RATE * TIME) / 100)
```

Task Description -5 (Re factoring – Enhancing Variable Naming and

Code Clarity)

- Task: Apply AI tools to improve code readability by renaming

  unclear variables and adding meaningful comments..

Instructions:

- Replace non-descriptive variable names with meaningful

  identifiers.
- Add inline comments where the logic may be unclear.
- Ensure program output remains the same.

Sample Code:

x = 50 y

= 30 z =

x + y

print(z)

Expected Output:

A Python program with clear variable names and explanatory comments

that improve readability without altering functionality.

Prompt and Code

```
# improve code readability by using meaningful variable names and adding comments to explain the purpose of each section of the code.
# Define two numbers to be added
number1 = 50
number2 = 30
# Calculate the sum of the two numbers
result = number1 + number2
# Print the result
print(result)
```